I2C Bus Monitor Plugin for Instrument Studio

This plugin provides an **I2C bus monitor, scanner, and transaction tool** inside **Instrument Studio 2025**, implemented with the **Measurement Plugin SDK** in **LabVIEW 2025**.

It is intended to be used as a **Small Panel** in Instrument Studio (a **Large Panel** for I2C Bus Monitor plugin can be useful for debugging).
It is most useful when used alongside Automotive Vision Capture Plugin in a **Large Panel**.
it is possible to use it in parallel with a FlexRIO Getting Started Example (GSE) with the instructions given in the I2C Bus Monitor startup dialog.

Key capabilities:

- **I2C Bus Monitor**: high-throughput capture into a monitor table (bulk FIFO reads + buffering).
- **I2C Transactions**: send a single transaction or run configuration scripts.
- **I2C Bus Scan**: probe for responding slave addresses (ACK).
- **Save CSV**: export the current monitor table to a CSV file.
- **Status + indicators**: running/streaming/busy plus error/status text.

Contents

Prerequisites

- **LabVIEW 2025 or newer**
  NI Drivers:
  - **NI-FlexRIO with Integrated I/O 2025 Q3 or newer**
- **Instrument Studio 2025 or newer**
- **Measurement Plugin SDK 3.5.0.2 or newer** (Installed through VI Package Manager)
- A supported NI PXIe GMSL/FPD-Link FlexRIO module that has a Serial Input channel (e.g. 4in4out or 8in variant):
  - **PXIe-1486**
  - **PXIe-1487**
  - **PXIe-1488**
  - **PXIe-1489**

Development Software Versions

The I2C Bus Monitor plugin is developed using these versions of the following software components:

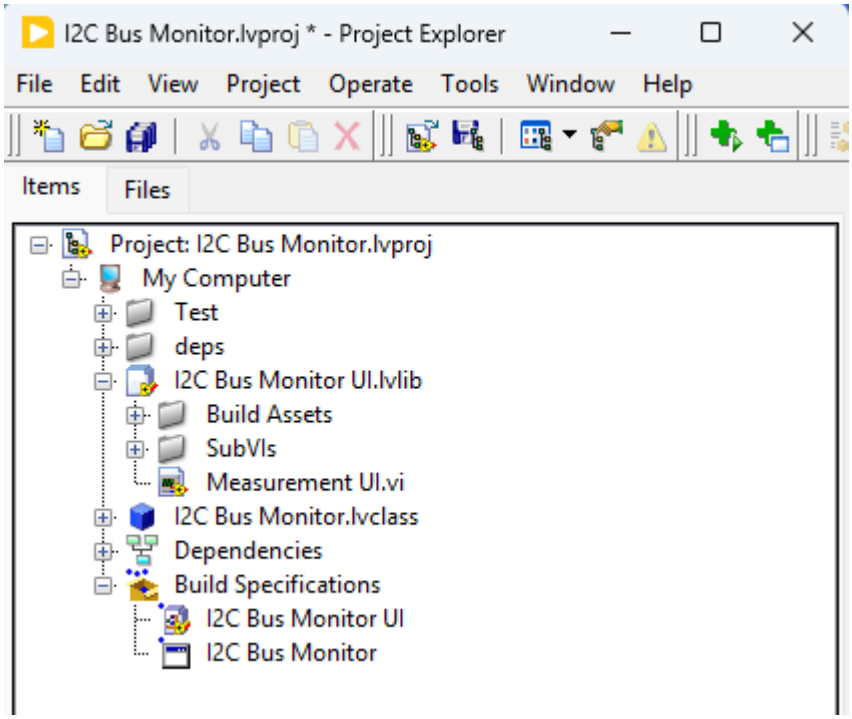
- **OS: Windows x64**

- **LabVIEW 2025 Q3**
  NI Drivers:
  - **NI-FlexRIO with Integrated I/O 2025 Q3**
- **Instrument Studio 2025 Q4**
- **Measurement Plugin SDK 3.5.0.2**

How to Build & Run the Plugin

1. **Open the project**

   Open `I2C Bus Monitor.lvproj` in **LabVIEW 2025**.



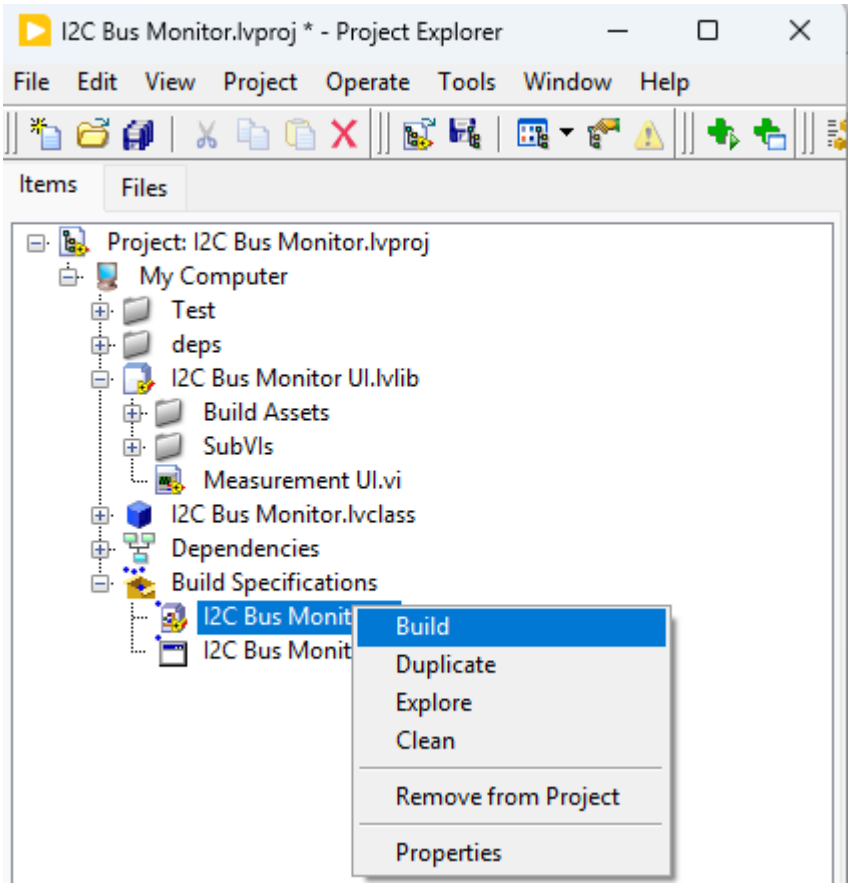
2. **Choose a workflow**

   ## Workflow 1: Debug from source (recommended for development)

   This workflow runs the service under LabVIEW so you can debug with breakpoints/probes.

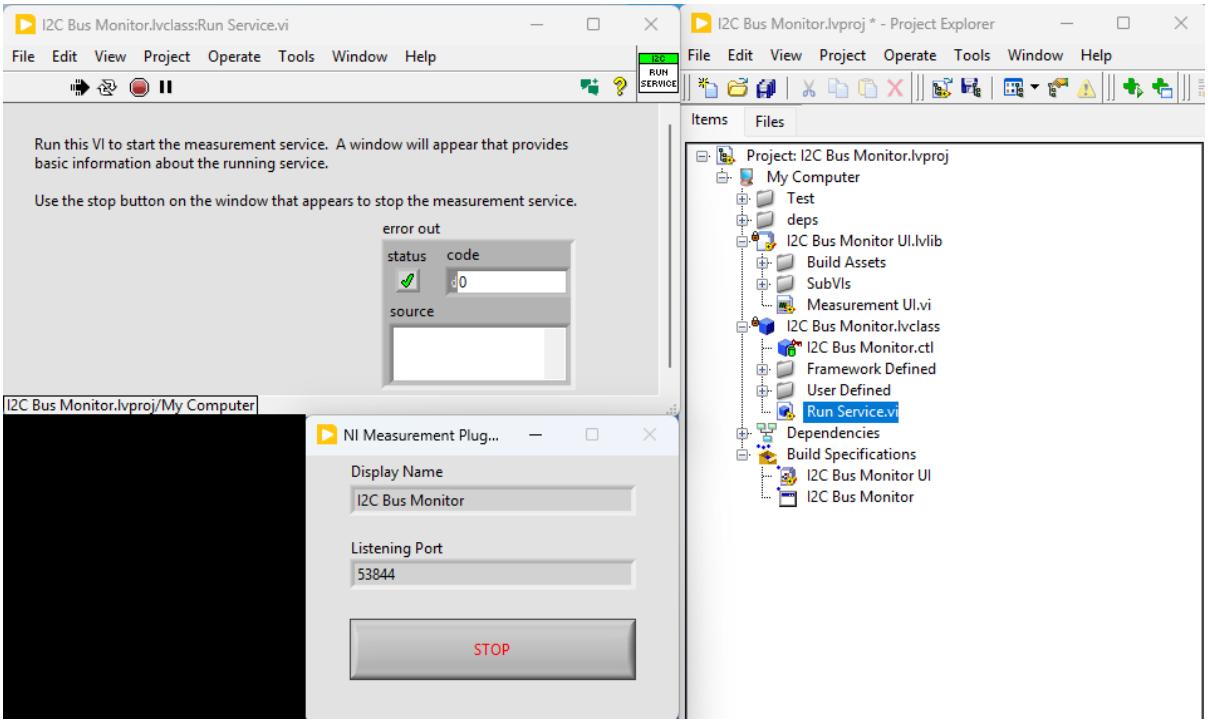
   1. Build the UI packed library:

      - `I2C Bus Monitor UI`

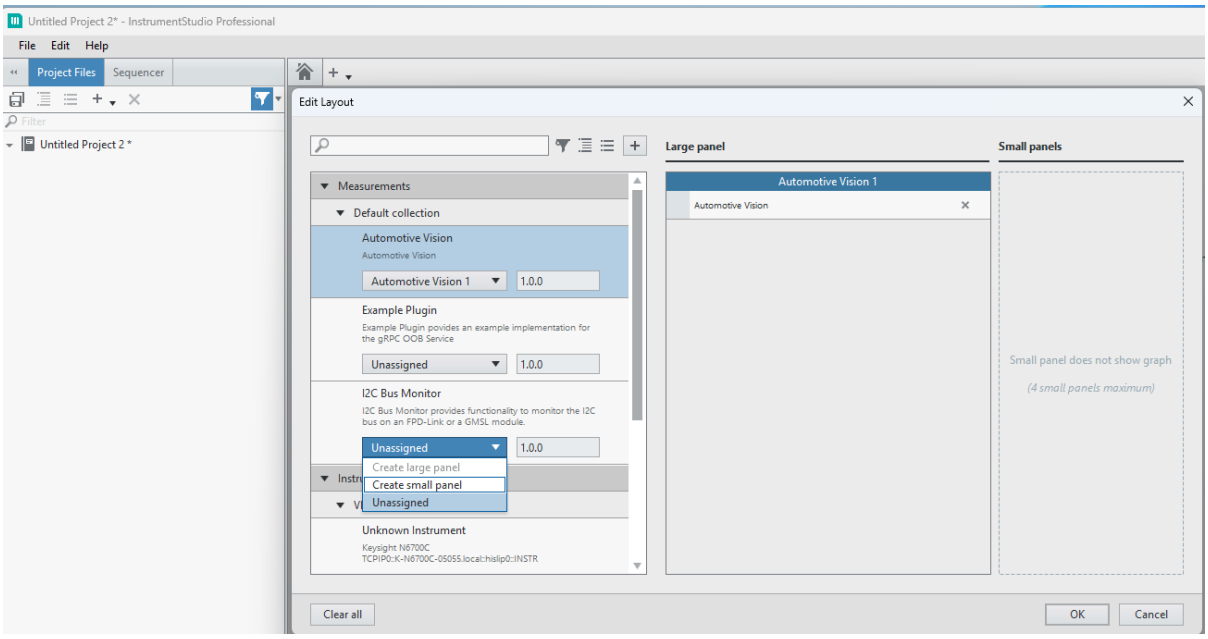


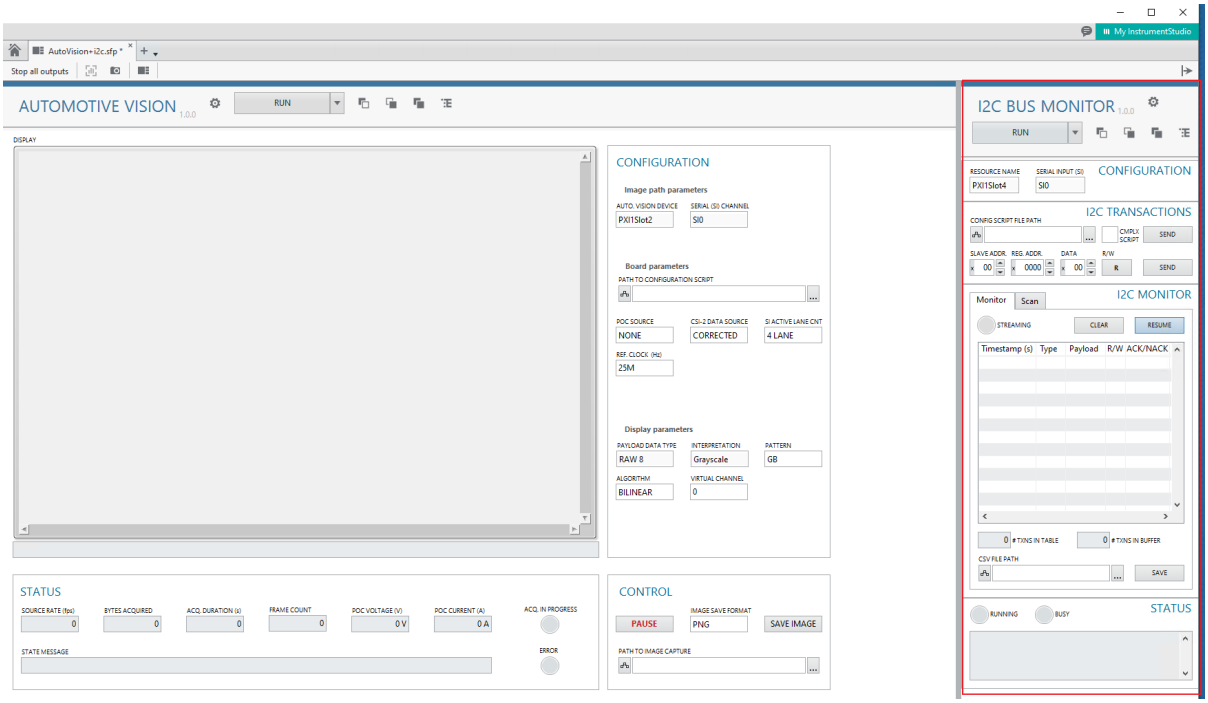
2. Open and run:

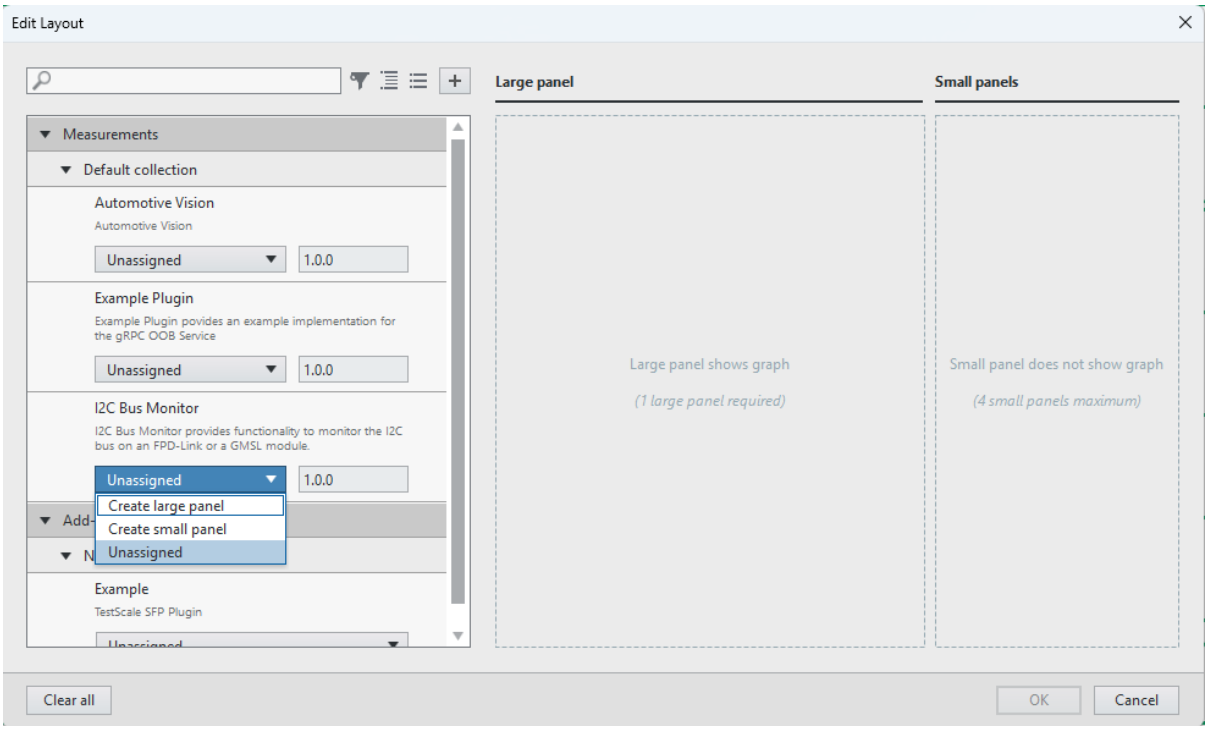
   - `I2C Bus Monitor\Run Service.vi`

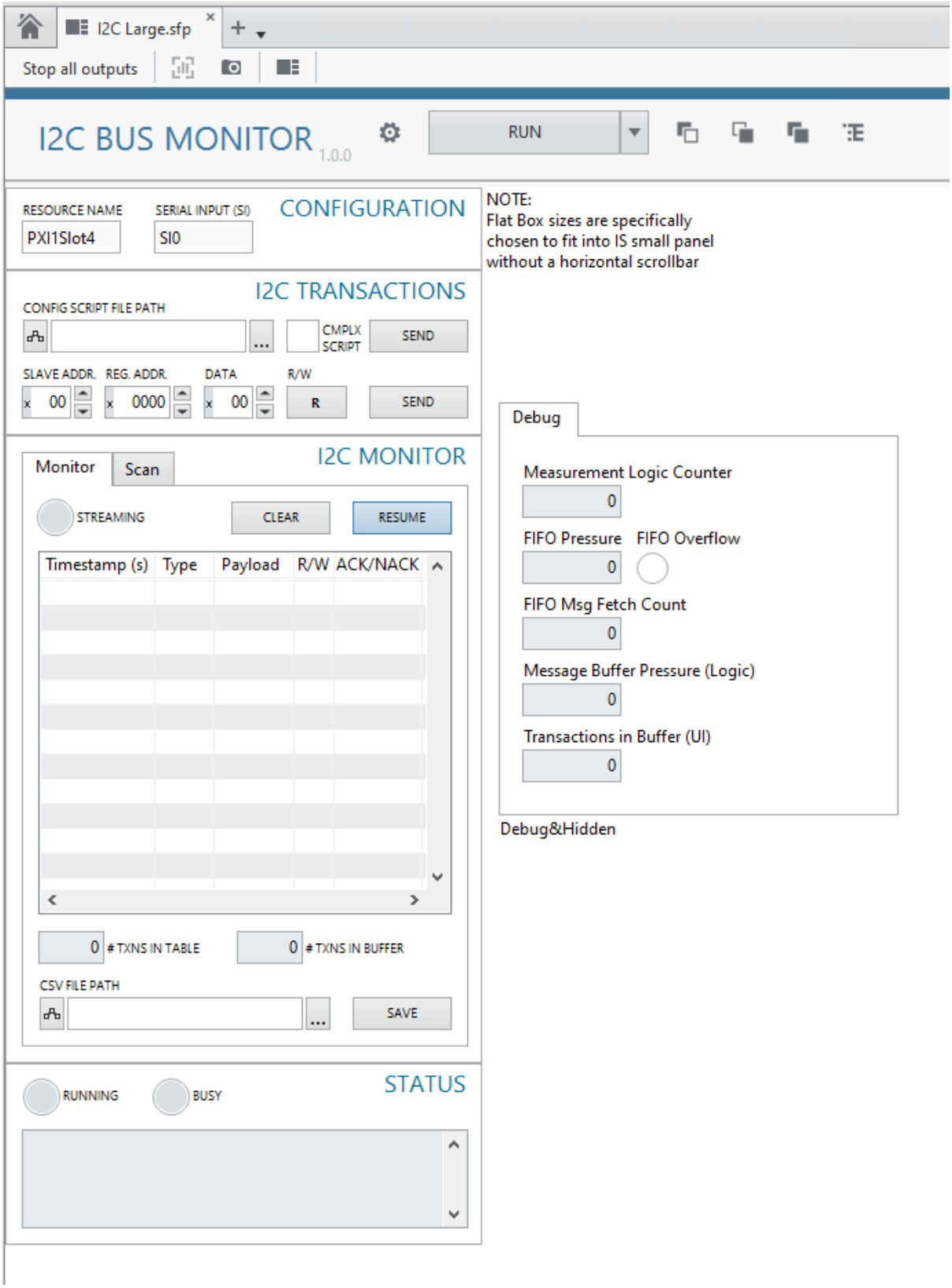


3. Open **Instrument Studio 2025**, create a **Manual Layout**, add a **Small Panel**, and select **I2C Bus Monitor**. (A **Large Panel** can be useful for debugging.)


Untitled Project 2* - InstrumentStudio Professional
File Edit Help
Project Files
Sequencer
Untitled Project 2 *
Edit Layout
Large panel
Small panels
Measurements
Default collection
Automotive Vision
Automotive Vision 1
1.0.0
Example Plugin
Example Plugin povides an example implementation for the gRPC OOB Service
Unassigned
I2C Bus Monitor
I2C Bus Monitor provides functionality to monitor the I2C bus on an FPD-Link or a GMSL module.
Create large panel
Create small panel
Unassigned
Unknown Instrument
Keysight N6700C
TCPIP0::K-N6700C-05055.local::hislip0::INSTR
Automotive Vision 1
Automotive Vision
Small panel does not show graph
(4 small panels maximum)
Clear all
OK
Cancel


AutoVision+i2c.sfp *
Stop all outputs
My InstrumentStudio
AUTOMOTIVE VISION
RUN
DISPLAY
CONFIGURATION
Image path parameters
AUTO. VISION DEVICE
PXI1Slot2
SERIAL (SI) CHANNEL
SI0
Board parameters
PATH TO CONFIGURATION SCRIPT
POC SOURCE
NONE
CSI-2 DATA SOURCE
CORRECTED
SI ACTIVE LANE CNT
4 LANE
REF. CLOCK (Hz)
25M
Display parameters
PAYLOAD DATA TYPE
RAW 8
INTERPRETATION
Grayscale
PATTERN
GB
ALGORITHM
BILINEAR
VIRTUAL CHANNEL
0
STATUS
SOURCE RATE (fps)
BYTES ACQUIRED
ACQ. DURATION (s)
FRAME COUNT
POC VOLTAGE (V)
POC CURRENT (A)
ACQ. IN PROGRESS
STATE MESSAGE
ERROR
CONTROL
PAUSE
IMAGE SAVE FORMAT
PNG
SAVE IMAGE
PATH TO IMAGE CAPTURE
I2C BUS MONITOR
RUN
RESOURCE NAME
PXI1Slot4
SERIAL INPUT (SI)
SI0
CONFIGURATION
I2C TRANSACTIONS
CONFIG SCRIPT FILE PATH
CMPLX SCRIPT
SEND
SLAVE ADDR.
REG. ADDR.
DATA
R/W
R
I2C MONITOR
Monitor
Scan
STREAMING
CLEAR
RESUME
Timestamp (s)
Type
Payload
R/W
ACK/NACK
TXNS IN TABLE
TXNS IN BUFFER
CSV FILE PATH
SAVE
STATUS
RUNNING
BUSY

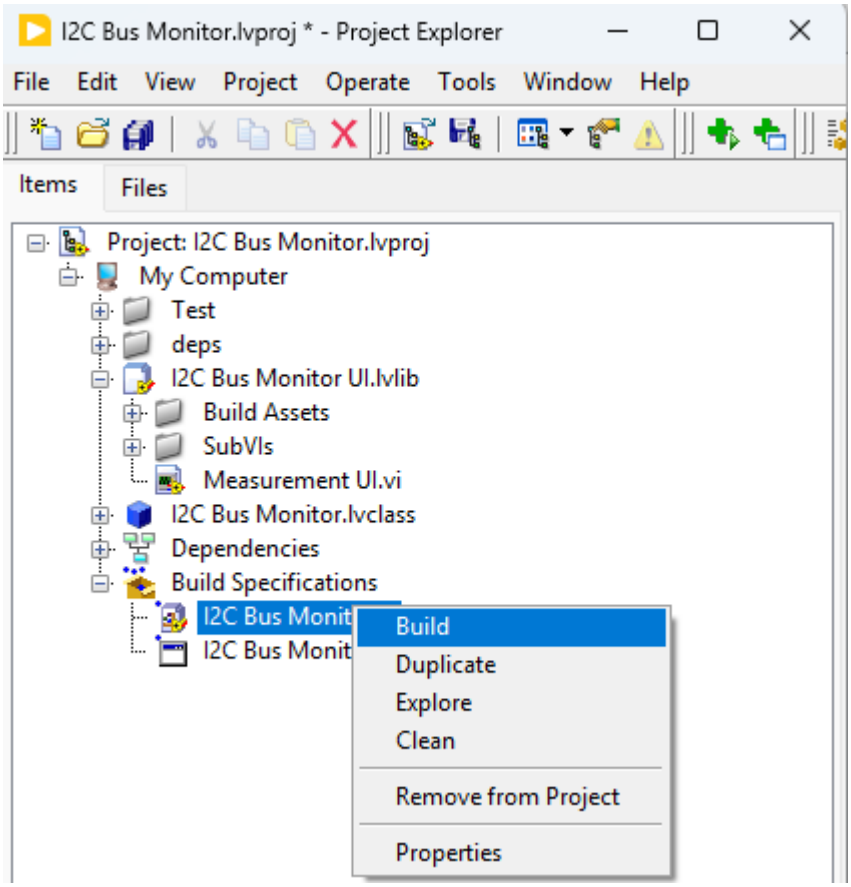

Edit Layout
Large panel
Small panels
Measurements
Default collection
Automotive Vision
Automotive Vision
Unassigned
1.0.0
Example Plugin
Example Plugin povides an example implementation for the gRPC OOB Service
Unassigned
1.0.0
I2C Bus Monitor
I2C Bus Monitor provides functionality to monitor the I2C bus on an FPD-Link or a GMSL module.
Unassigned
1.0.0
Create large panel
Create small panel
Unassigned
Example
TestScale SFP Plugin
Large panel shows graph
(1 large panel required)
Small panel does not show graph
(4 small panels maximum)
Clear all
OK
Cancel

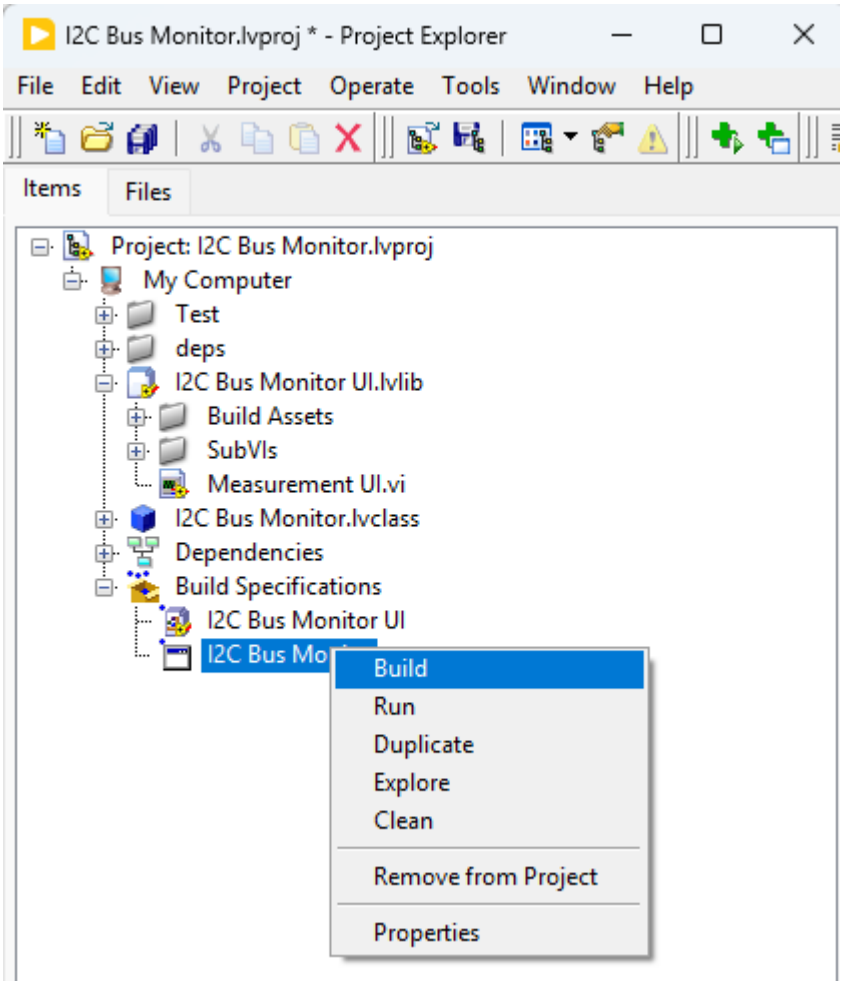


Workflow 2: Build + deploy for Instrument Studio (end-user use case)

This workflow builds the EXE and deploys the plugin so Instrument Studio can discover it.

1. Build both build specifications:
   - `I2C Bus Monitor UI` (packed library)
   - `I2C Bus Monitor` (EXE)

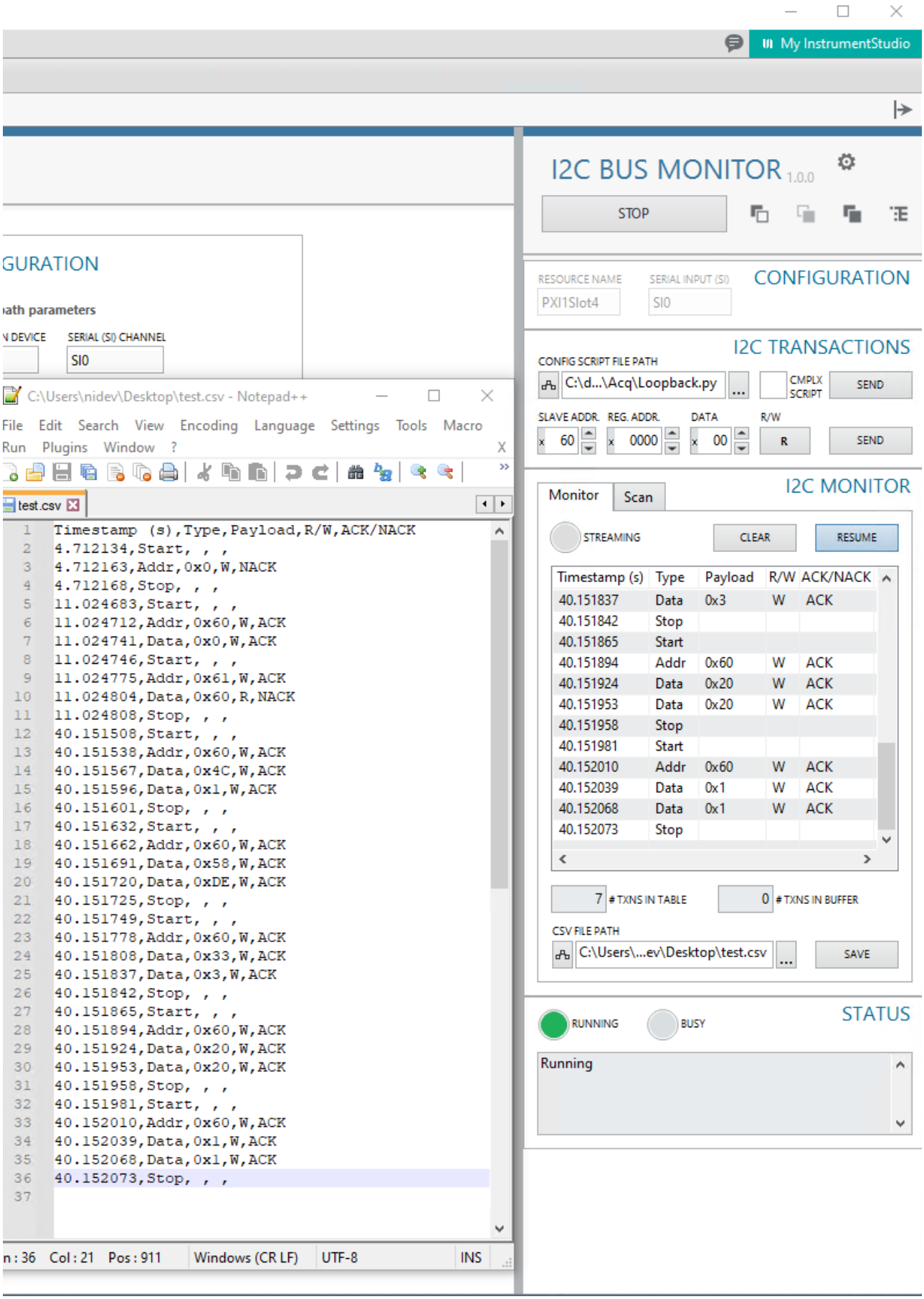


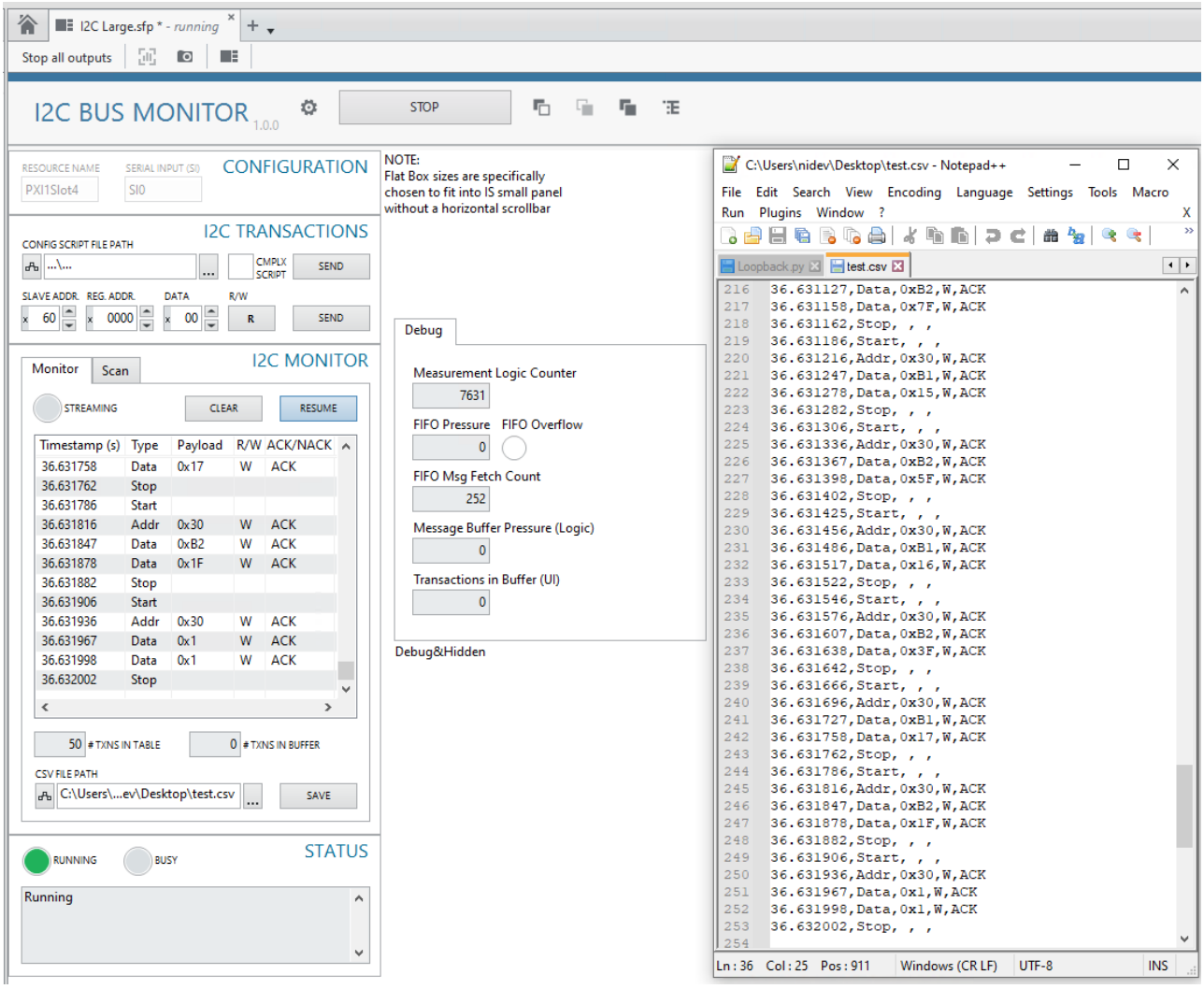


2. Launch (or restart) **Instrument Studio 2025**, then add the **I2C Bus Monitor** measurement.

Running the plugin in Instrument Studio

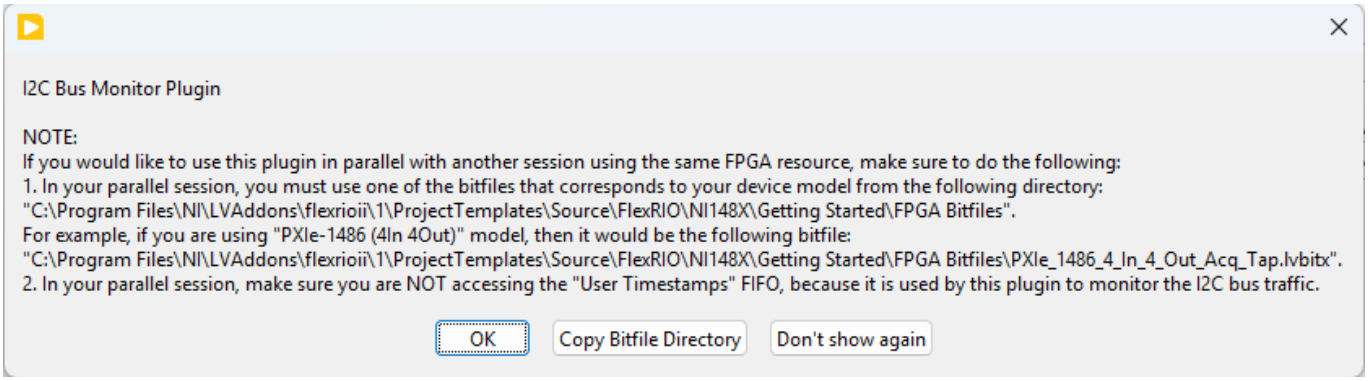
These screenshots show the plugin running after it has been added to a panel and shows a CSV file that has been saved and opened in notepad++:





Startup Dialog

On startup, the plugin displays a dialog describing how to use the plugin **in parallel** with another FPGA session.



The dialog includes:

- **Don’t show again** button
- **Copy Bitfile Directory** button (copies the folder path to the clipboard)

“Don’t show again” persistence

This is primarily intended for the built EXE:

- The EXE's `.ini` contains a key controlling whether the startup dialog appears.
- Default is **True** (show dialog).
- Clicking **Don't show again** sets it to **False**.

Bitfile directory

When using a parallel FPGA session, the parallel session needs to use bitfiles from this directory:

```
C:\Program
Files\NI\LVAddons\flexrioii\1\ProjectTemplates\Source\FlexRIO\NI148X\Getting
Started\FPGA Bitfiles
```

User Timestamps FIFO

When using a parallel FPGA session, the parallel session needs to make sure it is not accessing the User Timestamps FIFO, because it is used by this plugin for monitoring the I2C Bus traffic.

Configuration

The plugin UI populates:

- **Device Resource Name**
- **SI Channel** (serial channel)

Select the appropriate device and channel before starting the measurement.
Note that the plugin will throw an error if it doesn't find supported hardware on the system.

I2C Transactions

The plugin supports:

Send Single Transaction

Send a single I2C transaction from the UI.

Send Script

Run a configuration script.

Complex Script option (Python scripts)

For Python configuration scripts (`.py`) that contain more complex function calls, enable the **Complex Script** option.

If **Complex Script** is disabled, scripts are executed significantly faster than GSE.

I2C Bus Monitor

Monitor Table

This plugin is designed to handle I2C traffic efficiently by batching and buffering:

- **Logic side** reads the FPGA FIFO **in bulk** and buffers into a **logic-side internal buffer**.
- **UI side** fetches I2C transactions via gRPC **in bulk** and buffers into a **UI-side internal buffer**.
- The UI updates the **monitor table** by **dequeuing the UI buffer in bulk**.

Pause / Resume

Use **Pause** / **Resume** to control the *display* of captured traffic without stopping acquisition:

- **Pause** suspends updates to the **monitor table** so you can inspect a stable view.
- While paused, the plugin continues capturing; new transactions accumulate in the **UI-side buffer**.
- **Resume** re-enables table updates and drains buffered transactions into the monitor table.

Notes:

- **Save CSV** exports what is currently in the **monitor table** (what's displayed). If you are paused, buffered (not-yet-displayed) transactions are not included until you resume.
- If the bus is very active, leaving the plugin paused for a long time can grow the buffer.

Clear

Clears the **monitor table** content and flushes the **UI-side buffer**.

Monitor indicators

The UI provides:

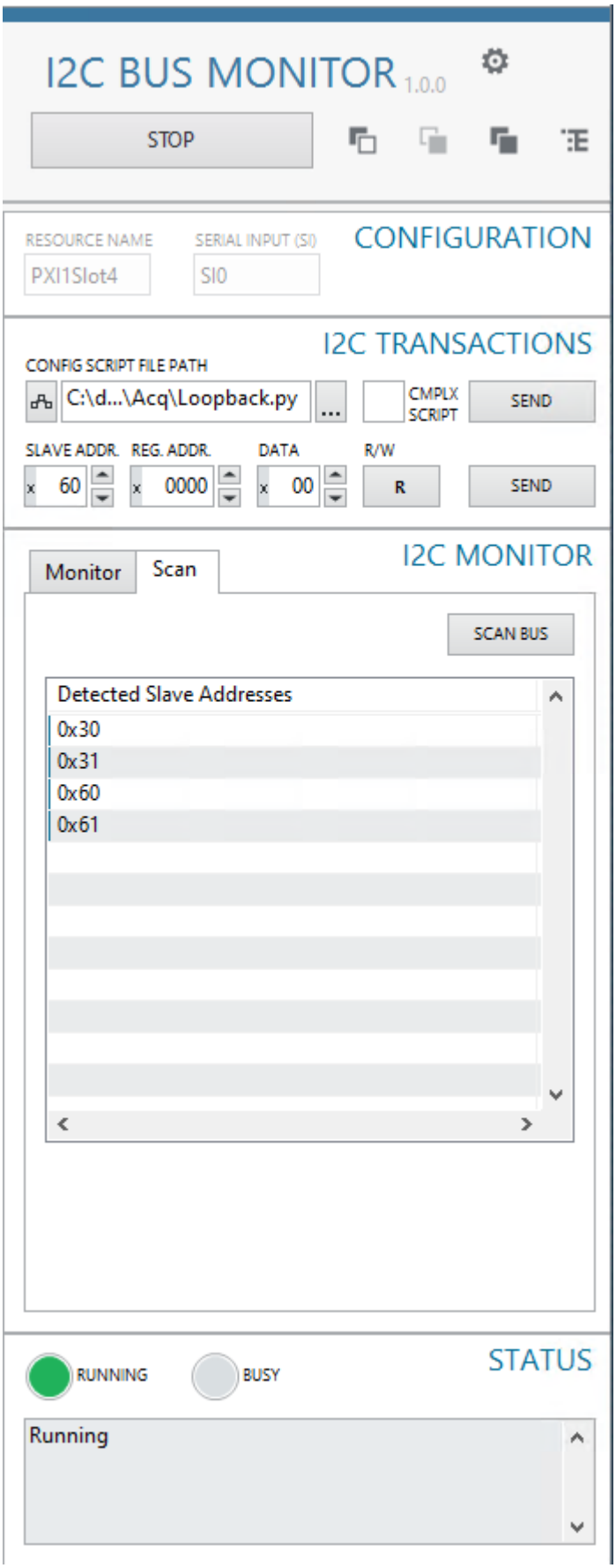
- **# TXNS IN TABLE**: number of transactions currently displayed in the monitor table
- **# TXNS IN BUFFER**: number of transactions currently waiting in the UI-side buffer
- **STREAMING**: turns on when a bulk of transactions is fetched by the ui side after being acquired by the logic side

Save CSV

Saves the current **monitor table** content to a **CSV** file at the path specified in the UI.

I2C Bus Scan

The scan feature:



- Returns **slave addresses** that respond with **ACK**.
- Flushes the FPGA FIFO after completing the scan to prevent flooding the bus monitor with scan transactions.

Status & Indicators

- **Status**: displays errors (and other status messages)
- **Running**: turns on when the **OOB gRPC connection** is connected
- **Busy**: turns on when **Running** is on and the logic side is hung for more than **500 ms**

Troubleshooting

- **No devices / channels listed**: verify drivers are installed and the device is visible in NI MAX.
- **No traffic shown**:
  - Confirm the correct device resource and SI channel are selected.
- **I2C Bus Scan doesn't show remote devices**:
  - If you are expecting I2C bus scan to discover a remote device (serializer/imager), this typically requires the Deserializer's I2C bus to be configured for I2C pass-through.
    For reference, see the the loopback configuration scripts that typically include such register configuration, these scripts are shipped with FlexRIO getting started examples. For Example, one can be found here for PXIe-1486 module:
    `C:\Program Files\NI\LVAddons\flexrioii\1\ProjectTemplates\Source\FlexRIO\NI148X\Getting Started\Host\Scripts\DS90UB954\Acq\Loopback.py`.
- **I2C Transactions:**
  - Typically you should see your device's Serial Input's (SI) Deserializer respond with ACK.
    (Typical power-up slave addresses for Deserializers are: `0x60` for FPD-Link, `0x90` for GMSL modules.)
    If there's no Serializer connected to the Deserializer or the Deserializer is not configured properly, the power-up slave address for the Serializer responds with NACK.
    (Typical power-up slave addresses for Serializers are: `0x30` for FPD-Link, `0x80` for GMSL modules).

Deployment

There are two common workflows depending on whether you are **debugging in LabVIEW** or doing a **real Instrument Studio deployment**.

Workflow 1: Debug the source code (recommended for development)

This workflow is convenient when you want to debug the service logic with normal LabVIEW tools (breakpoints, probes, highlighting).

1. In `I2C Bus Monitor.lvproj`, build the UI packed library:
   - `I2C Bus Monitor UI`
2. In LabVIEW, open and run:
   - `I2C Bus Monitor\Run Service.vi`
3. Open **Instrument Studio 2025**, add a panel, and select **I2C Bus Monitor**.

In this mode, the service runs from source under the LabVIEW environment, so you can debug `Measurement Logic.vi` directly.

Workflow 2: Deploy for Instrument Studio (real use case)

This is the intended deployment path for end users.

1. In `I2C Bus Monitor.lvproj`, build both build specifications:
    - `I2C Bus Monitor UI` (packed library)
    - `I2C Bus Monitor` (EXE)
2. The EXE build includes a post-build step that copies the plugin files into ProgramData so Instrument Studio can discover the measurement.
3. Launch (or restart) **Instrument Studio 2025**, then add the **I2C Bus Monitor** measurement.

Manual copy (fallback)

If you need to deploy manually (for example, customizing build outputs), copy the built plugin folder to:

```
C:\ProgramData\National Instruments\Plug-Ins\Measurements\I2C Bus Monitor
```